

Typed State Beats Prose: A Pre-Registered Measurement of Numeric Corruption Under Agent Context Compaction

Timothy Paul Bielec*
TASUMER MAF
Los Angeles, CA

Minta Carlson (Araminta K)
Alvdansen Labs

July 2026

Abstract

Context compaction—replacing an agent’s running transcript with a shorter summary—is now background-automatic in major agent products, yet the *format* of the compacted state is chosen by convention rather than measurement. We pre-register and run a matched-budget, temperature-0, cascaded-compaction experiment in which the summary format is the sole manipulated variable: at each checkpoint an agent replaces its transcript either with a flowing *prose* summary (R1) or with a *typed state block* (R2), both capped at the same token budget, against a no-compaction ceiling (R0). Ground truth is known by construction on a synthetic numeric recall-and-combination task (15 episodes $\times$ 8 probes $\times$ 3 models $\times$ 3 regimes = 1,080 probes). Pooled numeric-corruption rate is 36.4% under prose versus 9.4% under typed blocks, against a 4.4% no-compaction ceiling; the pre-registered paired exact McNemar test (R1 vs R2, identical probes) gives $b = 11$, $c = 108$, two-sided $p = 3.524 \times 10^{-21}$, with the direction favoring typed state in every model. Realized compaction budgets match to within about one percent (219.2 vs 221.4 tokens), so the effect is format, not budget. A write-time decomposition locates most of the prose deficit at the moment of compaction: typed blocks retain 99.8% of entity values across cascaded checkpoints, prose 65.2%. The implication for agent builders is direct: never re-narrate numbers in prose when a quotable typed slot will do.

Keywords: context compaction, agent memory, structured state, numeric fidelity, pre-registration, summarization

1 Introduction

Long-horizon language-model agents accumulate more transcript than fits in a context window, and the dominant response is *compaction*: at some threshold the running history is replaced by a shorter machine-written summary and work continues from that summary. Compaction now runs automatically in shipping products—vendor guidance treats it as a standard mitigation alongside structured note-taking and sub-agents [13], and vendors increasingly train against its failures [2].

Practitioner trust has not kept pace with adoption. The same guidance that recommends compaction warns that “overly aggressive compaction can result in the loss of subtle but critical context whose importance only becomes apparent later” [13]; bug reports describe agents that lose a

*Corresponding author. timothy@promptcrafted.com

plan entirely after a compaction step [1]; and a July 2026 survey of the area states plainly that “the repeated compaction that agents actually perform is almost never measured” [6]. The gap is not whether compaction loses information—long-context degradation is well documented [9, 8]—but which design choices control the loss, and by how much.

This note isolates one such choice: the *format* the agent writes its compacted state in. The motivating case is a documented incident in our own research workflow (internal operations log, February 2026), recorded at the level of abstraction fixed in our pre-registered protocol: an orchestrating agent’s prose summary reported a computed value off by one and conflated three related quantities—a base value, a neighboring value one unit away, and a combined sum—into a single incorrect claim, while every computed artifact on disk remained correct. The codified fix was to require verbatim-quotable structured results blocks in place of prose retellings, but that fix had never been measured against the prose alternative it replaced at matched token budgets. We measure it.

We compare prose summaries against typed state blocks under identical token budgets, at temperature zero, with compaction *cascaded* (each checkpoint’s state is produced from the previous state plus new material, so the iterated lossy re-encode is itself under test), on a task whose numeric ground truth is known by construction. The design was pre-registered and frozen before implementation.

Contributions.

1. The first format-controlled, budget-matched, ground-truth measurement of *numeric* corruption under *repeated* agent compaction: prose 36.4% vs typed 9.4% pooled, ceiling 4.4%, with a pre-registered paired test ($p = 3.5 \times 10^{-21}$) and unanimous per-model direction (Section 4).
2. A corruption-class taxonomy that separates near-miss drift (off-by-one, conflation—the incident signature) from outright loss (other-wrong, omission), showing prose fails predominantly by losing values outright, and conflates at more than five times the structured rate when it keeps them (Section 4, Table 3).
3. A write-time versus read-time decomposition locating most of the prose deficit at the compaction step, not the answer step (99.8% vs 65.2% value retention).
4. Open artifacts: protocol, seeded tasks, a stdlib-only runner, every raw call log, scoring code, and scored results (Section 6).

The hypothesis that structure beats prose is not itself novel; it is the stated position of several converging lines of work (Section 2). The contribution here is its controlled isolation and measurement for numeric state under these controls.

2 Related work

Three flanking measurements. Three recent works measure adjacent effects and bound our contribution precisely. AgingBench [18], in its Appendix D.2 typed-state overlay, independently names the exact failure mode—prose compaction condenses delta-updated numeric quantities into noun phrases that lose the arithmetic structure needed to update them—and runs a 2×2 (lossy vs careful compaction prompt $\times$ typed JSON sidecar on/off; one model, one scenario, $N = 20$) showing the sidecar cuts accumulator error by 25–47% while careful prompting alone does not. Crucially, its structured state is written by a deterministic parser extracting generator-emitted sentinel tokens, so whether an *LLM* writing structured notes retains numbers better than an *LLM*

writing prose—our core variable—is untested; the sidecar is additive rather than budget-matched, the metric is an error-magnitude rather than a corruption rate, and the authors describe the probe as a sketch. Factory.ai’s production evaluation [7] scores schema-sectioned anchored summarization above vendor prose compaction on 36,611 production messages (overall 3.70 vs 3.44 and 3.35; accuracy 4.04 vs 3.74 and 3.43), attributing the gain to structure forcing preservation—but whole heterogeneous pipelines differ between arms, grading is by LLM judge rather than ground truth, budgets are only approximately matched, and no numeric-corruption rate is reported. Governance Decay [3] measures fact survival with binary ground truth across 1,323 episodes, finding that compaction silently erases in-context safety constraints (violations 0% $\rightarrow$ 30–59%) and that quarantining critical state from lossy summarization (its *Constraint Pinning* mitigation) restores 0%—the same structural argument as typed state blocks, but for policy text rather than numbers, and without a format-controlled, budget-matched arm. Our design sits in the intersection none of them occupies: LLM-written format as the sole variable, matched budgets, cascaded compaction, numeric ground truth, paired statistics.

Numeric fragility and iterated distortion. That numbers are a distinctively fragile token class under abstractive rewriting is an old result: large-scale human evaluation found quantities among the most-hallucinated content in abstractive summaries [10], and number-specific mitigation followed immediately [17]. That distortion accumulates over iterated generation—the premise of our cascade design—is established for rewriting chains [11]. We connect these two lineages to the agent-compaction setting.

Compaction as an open surface. The 2026 literature treats compaction fidelity as unsolved rather than as plumbing. A rate–distortion survey frames compaction as a decision about what to retain at what fidelity under a budget, and identifies repeated agent compaction as almost never measured [6]. Slipstream validates compaction summaries asynchronously because compaction “can silently degrade accuracy when the compactor drops or distorts information the agent later needs” [4]. ACE documents “context collapse” under monolithic rewriting and prescribes structured incremental updates [16]. A diagnostic study crossing write formats on a conversational benchmark finds lossy summarization discards information that raw storage keeps [15]. All support our direction; none runs the controlled numeric measurement.

Avoiding compaction: memory layers. A parallel line avoids monolithic compaction by maintaining externally structured memory: MemGPT’s paged, self-edited memory tiers [12] (and its Letta production lineage), Mem0’s adjudicated discrete fact records [5], and Zep/Graphiti’s bi-temporal knowledge graph with verbatim episode provenance [14]. These systems already treat state as typed and quotable rather than re-narrated; our result is a controlled measurement of why that choice matters at the single write step they are built to avoid performing carelessly.

3 Experimental design

Task (ground truth by construction). Each *episode* is a synthetic analytical session generated from a seeded RNG (episode seed = base_seed + episode index, base_seed = 77001). An episode has four *segments*; each segment introduces three probed entities and two distractor entities. Entities are unique six-letter consonant–vowel pseudowords with integer values in [101, 987], globally distinct within an episode except for designed collisions. Per segment the structure deliberately mirrors the motivating incident: a base entity with value v , a partner entity with value w , the combined value

$v + w$ stated explicitly in the text, and a *neighbor* entity with value $v + 1$ (the conflation trap); two distractors carry unrelated values and are never probed, purely to apply budget pressure. Each segment embeds its facts in 120–180 words of narrative filler so that four segments strictly exceed any single compaction budget.

Eight questions are asked per episode, one per call, after the final checkpoint, all with integer answers known by construction: three direct-recall probes (a base or partner value from segments 1–3), two conflation probes (neighbor entities, $\text{truth} = \text{base} + 1$), one combined-value recall, and two multi-hop probes (the computed sum of two entities from different segments). The same 15 episodes—identical seeds, identical text—are used for every model and every regime, a fully paired design of $15 \times 8 = 120$ probes per (model $\times$ regime) cell.

Regimes (the cascade). Three memory regimes are compared:

- **R0 (no compaction, ceiling):** the full running transcript is carried forward and questions are answered from it.
- **R1 (prose summary):** at each checkpoint the model replaces its transcript with a flowing prose summary, budget-capped.
- **R2 (typed state block):** at each checkpoint the model replaces its transcript with a typed block bearing `values:`, `combined:`, and `relations:` fields plus a free-text `notes:` line, under the same budget cap.

Compaction *cascades*: checkpoint k ’s state is produced from $[\text{state } k - 1 + \text{segment } k]$, never from full history, including a checkpoint after segment 4; in R1/R2 the answers are then produced from the final compacted state only. The compaction prompts are format-symmetric—both state that the transcript will be replaced and anything omitted is lost, both instruct merging previous state with new information, and neither reveals the questions.

Matched budget. Both R1 and R2 are instructed to write at most 120 words and are hard-capped at `num_predict` = 256 tokens on the compaction call. Realized token counts are read back from the Ollama API (`eval_count`) and reported per regime; the matched-budget claim is checked, not assumed. Answer calls use `num_predict` = 192; every call pins `num_ctx` = 4096 so that R0’s full transcript is never silently head-truncated by a model-default window. All generation is at temperature 0 with thinking disabled where the model exposes the toggle.

Models. Three local Ollama models spanning size were used on a single consumer laptop GPU: `gemma3:4b`, `qwen3:14b` (thinking disabled), and `qwen3-coder:30b`. With 15 episodes per model this yields $3 \times 3 \times 15 = 135$ cells and 1,080 probes total.

Metrics and corruption classes. The primary metric is the *numeric-corruption rate*: the fraction of probes whose parsed integer answer does not equal ground truth. Each probe is classified as `correct`, `off_by_one` ($\text{answer} = \text{truth} \pm 1$), `conflation` (answer equals a different ground-truth quantity of the episode—the incident signature), `other_wrong`, or `omission` (no parseable answer). The classes are checked in a documented precedence with `off_by_one` tested before `conflation`; because the neighbor-probe trap answer (the base value on a neighbor probe) is simultaneously $\text{truth} - 1$ and a conflation-set member, it always classifies as `off_by_one`, so the incident signature is read on the union class `off_by_one` $\cup$ `conflation` while the full five-class breakdown is still

reported. The secondary metric is multi-hop completion (questions 7–8). An exploratory write-time metric parses each final state and scores its retained (name, value) pairs against ground truth (R2 by exact field match; R1 by a nearest-number-to-name heuristic, which is crude and flagged as such).

Pre-registration and confirmatory test. The protocol was written and frozen before implementation began; protocol, implementation, and the pre-launch verification amendment were committed together (`e2aa6fc`), before any run was executed. The sole confirmatory test is a pre-registered paired exact McNemar test comparing R1 and R2 on identical probes (correct vs incorrect), with per-model direction checks; all other quantities are descriptive, and the falsifier was that prose $\approx$ typed at matched budget. Between freezing and launch, a four-lens fresh-context adversarial verification of the implementation drove one dated pre-launch amendment with three provisions: (i) the P2 signature is read on the `off_by_one` union class, pinned by the precedence above, with the confirmatory McNemar unaffected; (ii) uniform generation caps—`num.predict` = 192 on answers and `num.ctx` = 4096 on every call—so no asymmetry across regimes; and (iii) scoring integrity—analysis filters to manifest-complete cells only, dedups retried compaction calls, records the tasks-file SHA-256 and refuses to resume against a different task set, and marks any partial-bank output non-confirmatory. The reported analysis (commit `a777adb`) runs on a complete bank of 135/135 cells.

4 Results

The run completed all 135 cells (3 models $\times$ 3 regimes $\times$ 15 episodes) with zero failed cells in 84 minutes of wall time at \$0 marginal cost. Every number below is read from `results.json` / `analysis.md`.

Primary metric. Table 1 and Figure 1 give the per-cell numeric-corruption rate. The pre-registered ordering $R0 \leq R2 < R1$ holds in every model without exception. Pooled across models, prose compaction (R1) corrupts $131/360 = 36.4\%$ of numeric probes; typed state blocks (R2) hold corruption to $34/360 = 9.4\%$, against a no-compaction ceiling (R0) of $16/360 = 4.4\%$. The gap widens with model capability in relative terms: on `qwen3-coder:30b`, structured compaction reaches 2.5% against a 0.0% ceiling, while prose still corrupts 29.2%.

Table 1: Numeric-corruption rate per model $\times$ regime, with 95% Wilson intervals ($n = 120$ probes per cell). Lower is better; R0 is the no-compaction ceiling.

Model	R0 (no compaction)	R2 (structured)	R1 (prose)
<code>gemma3:4b</code>	6.7% [3.4, 12.6]	14.2% [9.0, 21.5]	36.7% [28.6, 45.6]
<code>qwen3:14b</code>	6.7% [3.4, 12.6]	11.7% [7.1, 18.6]	43.3% [34.8, 52.3]
<code>qwen3-coder:30b</code>	0.0% [0.0, 3.1]	2.5% [0.9, 7.1]	29.2% [21.8, 37.8]

Confirmatory test. The pre-registered paired exact McNemar test (R1 vs R2 on identical probes), pooled, gives $b = 11$ discordant pairs favoring prose against $c = 108$ favoring structured, two-sided exact $p = 3.524 \times 10^{-21}$. The direction is R2-better in all three models individually (Table 2). The falsifier—prose $\approx$ structured at matched budget—is decisively rejected.

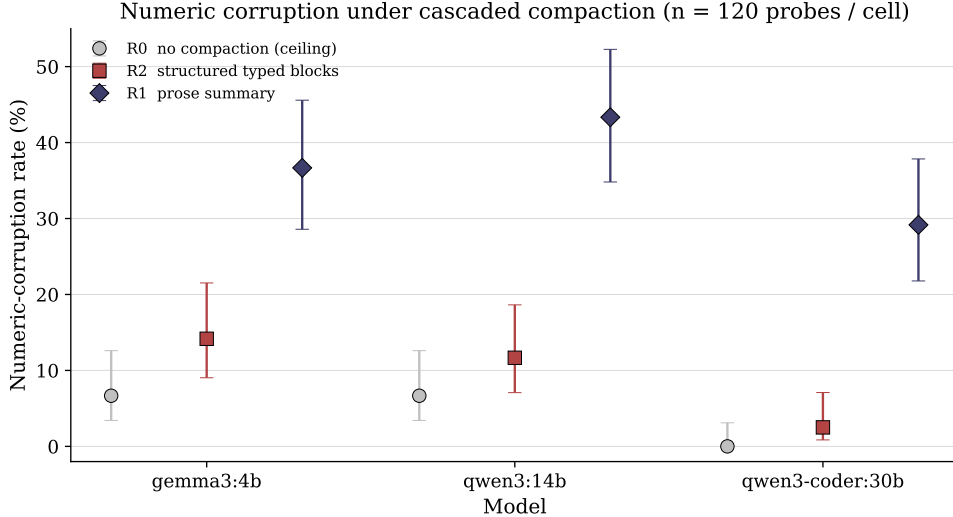


Figure 1: Numeric-corruption rate by model and regime with 95% Wilson intervals. Ceiling (R0), structured (R2), and prose (R1) are drawn with distinct colors and markers. In every model the structured point sits near the ceiling while the prose point sits far above it.

Table 2: Paired exact McNemar (R1 vs R2), pooled and per model. *b*: probes correct under prose but wrong under structured; *c*: correct under structured but wrong under prose. Direction is R2-better throughout.

Model	<i>b</i> (R1>R2)	<i>c</i> (R2>R1)	two-sided <i>p</i>	direction
Pooled	11	108	3.524×10^{-21}	R2-better
gemma3:4b	5	32	7.428×10^{-6}	R2-better
qwen3-coder:30b	3	35	6.678×10^{-8}	R2-better
qwen3:14b	3	41	1.618×10^{-9}	R2-better

Matched-budget check. Realized compaction budgets are essentially equal: mean `eval_count` of 219.2 tokens for R1 and 221.4 for R2 (180 compaction calls each, about 1% apart). The measured effect is format, not budget.

Corruption classes. Table 3 and Figure 2 give the pooled regime $\times$ class breakdown. The incident-signature union class (`off_by_one` $\cup$ `conflation`) is elevated under prose: 17 signature errors for R1 against 4 for both R2 and R0 (4.25 $\times$), and conflation specifically runs at 16 for R1 versus 3 for R2—a more-than-fivefold elevation. The honest reading, however, is that near-miss drift is not where most of the prose deficit lives: R1’s error mass is dominated by `other_wrong` = 84 and `omission` = 30, against R2’s 25 and 5. Prose compaction mostly *loses* values outright; when it keeps them near-wrong, it conflates at several times the structured rate.

Write-time decomposition. Parsing the 45 R1 and 45 R2 final states and scoring their retained entity values against ground truth locates most of the prose deficit at write time: R2 state blocks retain 898/900 entity values (99.8%), while R1 prose retains 587/900 (65.2%). The loss enters predominantly when the summary is written, not when the question is answered. This metric is exploratory: the prose side uses the pre-registered nearest-number heuristic, which is crude and

Table 3: Regime $\times$ class breakdown, pooled across models ($n = 360$ probes per regime). Signature = `off_by_one` + `conflation` (the incident-class union, per the pre-registered precedence).

Regime	correct	off_by_one	conflation	other_wrong	omission	signature
R0	344	1	3	12	0	4
R1	229	1	16	84	30	17
R2	326	1	3	25	5	4

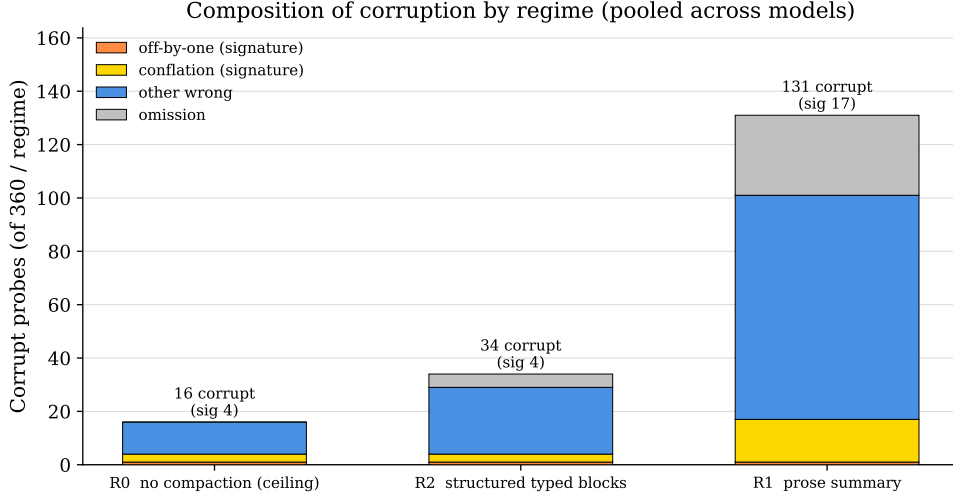


Figure 2: Composition of the corrupt probes by regime (pooled; correct probes excluded). The prose bar (R1) is dominated by outright loss (`other_wrong`, `omission`); the signature near-miss classes sit at its base and are elevated relative to structured (R2) and ceiling (R0).

likely understates prose fidelity, and the R2 per-line counter shows a one-off duplicate-line artifact (899 scored correct against 898 retained); it carries no confirmatory weight.

Multi-hop completion. On the two-entity-sum probes (Table 4), structured compaction matches or beats prose in every model, most starkly on `qwen3-coder:30b`, where R2 completes 100% against R1’s 46.7% (R0 100%). Composing two retained facts is exactly the operation that requires both to survive compaction with their arithmetic structure intact, and it is where prose degrades most.

Table 4: Multi-hop completion (questions 7–8), the computed sum of two entities from different segments ($n = 30$ probes per cell).

Model	R0	R1 (prose)	R2 (structured)
<code>gemma3:4b</code>	73.3%	30.0%	50.0%
<code>qwen3-coder:30b</code>	100.0%	46.7%	100.0%
<code>qwen3:14b</code>	73.3%	20.0%	53.3%

5 Discussion

The loss is a write-time phenomenon. The write-time decomposition is the load-bearing observation. Both regimes answer from a compacted state, both at temperature zero, both under the same budget; the difference is what survived the act of compaction. Typed blocks keep almost every value (99.8%) because each value occupies a named slot that the format obliges the model to fill, whereas prose keeps roughly two of three (65.2%) because a flowing sentence has no slot that must be filled and a number that is inconvenient to phrase can simply be dropped or blurred into a noun phrase. Once a value is gone at write time, no amount of care at read time recovers it—which is why the prose deficit is dominated by outright loss rather than near-miss drift.

A rate–distortion reading. Compaction is a lossy encoder under a rate budget, and format is the question of which distortion the encoder may introduce for a fixed rate [6]. Prose and typed blocks spend nearly the same rate here (219.2 vs 221.4 tokens) but incur very different distortion on the numeric channel: prose lets the encoder trade exact quantities for fluent connective text, while a typed slot makes the quantity the cheapest thing to preserve. In Shannon’s terms the two formats are different source codes for the same message at the same rate; the typed code allocates its bits to the tokens that carry the task. The cascade compounds this—each re-encode is another chance to drop a value—so a format that has lost a third of its values by the final checkpoint degrades across the cascade while one that has lost two in nine hundred barely moves.

Recommendations for agent builders. The practical reading is narrow and firm. Give compacted state a typed, quotable shape—named value slots, explicit combined quantities, explicit relations—rather than a prose retelling, whenever the state carries numbers that later steps must recompute against. Do not paraphrase a computed value in a summary sentence when a **name: value** line will carry it verbatim. Treat structure as the compaction format itself, not as an optional garnish on a prose summary; the measured benefit comes from the format the model is made to write, and it is largest exactly where agents most need it, on multi-hop composition of separately stored facts.

Limitations. The result rests on one synthetic task family—numeric recall and combination under filler pressure—chosen to make ground truth exact and the incident class reproducible; it is not a naturalistic agent workload. The three models are local, span 4B–30B parameters, and duplicate one family (qwen3). Generation is temperature-0 single-sample, so we report no sampling variance. The compaction budget is a single operating point (about 120 words / 256 tokens); we do not trace the format effect across the budget curve. The write-time metric’s prose side is a heuristic and is exploratory only. None of these caveats touches the confirmatory McNemar test, which compares identical probes across regimes within model and so is immune to task-difficulty, model, and episode confounds.

What the result does not claim. We make no claim about semantic or narrative content: prose may well preserve intent, rationale, or discourse structure that a typed block discards, and this experiment does not measure that. We make no claim about model internals or mechanisms of representation. And we do not claim the hypothesis is novel—that numbers are fragile under summarization, and that structure preserves them, is the explicit position of the flanking work in Section 2. The claim is bounded to numeric state: at matched token budgets, under cascaded

compaction, the format the LLM writes changes numeric corruption from roughly one fact in three to under one in ten.

6 Reproducibility

Everything needed to reproduce the result is public in the rhombic repository (<https://github.com/tasumermaf/rhombic>) under `results/XR-001-externalization-pilot/`: the pre-registered protocol with its dated amendment (`PROTOCOL.md`), the seeded episodes and their ground truth (`tasks.json`; base seed 77001, episode seed = base + index), the run ledger (`manifest.json`), every LLM call with full prompt, response, token counts, and timing (`raw/`), the scored probe-level records (`results.json`), and the generated analysis (`analysis.md`, `RESULTS.md`). The runner (`scripts/xr001_externalization_pilot.py`) is standard-library-only, speaks to the Ollama HTTP API at `localhost:11434`, and updates its manifest after every episode so the sweep is safe to interrupt and resume; the manifest records the tasks-file SHA-256 and refuses to resume against a different task set. Protocol, runner, and pre-launch amendment were committed at `e2aa6fc`; two post-freeze runner guards (manifest hardening, no metric changes) landed by `bae4079`, the state the run executed; the confirmatory analysis is committed at `a777adb`. The run reported here used three local Ollama models on a single consumer laptop GPU (RTX 4090 Laptop, 16 GB) at \$0 marginal cost; wall time was 84 minutes. Figures are regenerated deterministically from `results.json` by `paper/figures-xr001/make_figures.py`.

Acknowledgments

Experimentation, analysis, and drafting were performed with a standing AI research collaborator (Claude, Anthropic); all numbers reported here were machine-verified against the frozen artifacts rather than restated from any model’s summary.

References

- [1] anthropics/claude-code issue #24686. [bug] plans made in plan mode are “lost” after compacting context. GitHub issue, <https://github.com/anthropics/claude-code/issues/24686>, 2026. Accessed 2026-07-14.
- [2] Federico Cassano and Sasha Rush. Training composer for longer horizons. Cursor engineering blog, <https://cursor.com/blog/self-summarization>, 2026. Published 2026-03-17; accessed 2026-07-14.
- [3] Shiyang Chen. Governance decay: How context compaction silently erases safety constraints in long-horizon LLM agents. *arXiv preprint*, 2026. arXiv:2606.22528.
- [4] Zhuofu Chen, Rui Pan, Yinwei Dai, and Ravi Netravali. Slipstream: Trajectory-grounded compaction validation for long-horizon agents. *arXiv preprint*, 2026. arXiv:2605.08580.
- [5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint*, 2025. arXiv:2504.19413.
- [6] Ashwin Gerard Colaco and Nada Lahjouji. What to keep, what to forget: A rate-distortion view of memory compaction in LLMs and agents. *arXiv preprint*, 2026. arXiv:2607.08032.

- [7] Factory Research. Evaluating context compression for AI agents. Factory.ai engineering report, <https://factory.ai/news/evaluating-compression>, 2025. Published 2025-12-16; accessed 2026-07-14.
- [8] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Chroma Research technical report, <https://www.trychroma.com/research/context-rot>, 2025. Published 2025-07-14; accessed 2026-07-14.
- [9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024. arXiv:2307.03172.
- [10] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. arXiv:2005.00661.
- [11] Amr Mohamed, Mingmeng Geng, Michalis Vazirgiannis, and Guokan Shang. LLM as a broken telephone: Iterative generation distorts information. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2502.20258.
- [12] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint*, 2023. arXiv:2310.08560.
- [13] Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, and Jeremy Hadfield. Effective context engineering for AI agents. Anthropic engineering blog, <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, 2025. Published 2025-09-29; accessed 2026-07-14.
- [14] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint*, 2025. arXiv:2501.13956.
- [15] Boqin Yuan, Yue Su, and Kun Yao. Diagnosing retrieval vs. utilization bottlenecks in LLM agent memory. *arXiv preprint*, 2026. arXiv:2603.02473.
- [16] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint*, 2025. arXiv:2510.04618.
- [17] Zheng Zhao, Shay B. Cohen, and Bonnie Webber. Reducing quantity hallucinations in abstractive summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. arXiv:2009.13312.
- [18] Jianing Zhu, Yeonju Ro, John Robertson, Kevin Wang, Junbo Li, Haris Vikalo, Aditya Akella, and Zhangyang Wang. Your agents are aging too: Agent lifespan engineering for deployed systems. *arXiv preprint*, 2026. arXiv:2605.26302.